Week 3 — Function Calling, Tools, and LLMs That Can Act

12 March 2026

Part 1 — Three Little Pigs Demo

Configuration

The demo was configured by creating a `.env` file in the `function-calling/` folder using the credentials from `apikey.md`:

```
OPENAI_API_KEY=sk-...
OPENAI_API_ENDPOINT=https://dashscope-intl.aliyuncs.com/compatible-mode/v1
MODEL=qwen3.5-122b-a10b
```

The script loads these values at start-up with `python-dotenv` and constructs the `OpenAI` client:

```
load_dotenv()
client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    base_url=os.getenv("OPENAI_API_ENDPOINT"),
)
```

The program is run with:

```
$ cd week-03/function-calling
$ uv run python three_pigs_function_calling.py
```

The interactive menu offers two options — chat without tools (option 1) and chat with tools (option 2).

Scenario 1 — Without Function Calling

With tools **disabled** the model receives only the system prompt and the user message. It must respond with text alone.

```
$ python three_pigs_function_calling.py

  Choose option (1/2/q): 1

  You (wolf): Open the door or I will blow your house down.

  [API Request sent]
    model   : qwen3.5-122b-a10b
    messages: [ system, user ]    ← no "tools" key

  [API Response received]
    finish_reason: stop
    content: "My bricks are too strong for your tricks, even if I
              am shaking with fear! I have already called the
```

```
            hunter—he will arrive in time to stop you."

Pig: My bricks are too strong for your tricks, even if I am
     shaking with fear! I have already called the hunter—he
     will arrive in time to stop you.
```

The pig mentions the hunter verbally, but **no real action is taken** — the model can only produce text. `finish_reason: stop` confirms the model ended the turn with prose.

Scenario 2 — With Function Calling

With tools **enabled** the same prompt produces a completely different API response.

```
Choose option (1/2/q): 2

You (wolf): Open the door or I will blow your house down.

[API Request sent — Round 1]
  model   : qwen3.5-122b-a10b
  messages: [ system, user ]
  tools   : [ call_hunter ]     ← tool schema included

[API Response received — Round 1]
  finish_reason: tool_calls     ← model did NOT write text
  tool_calls[0]:
    name      = 'call_hunter'
    arguments = {
      "urgency": "emergency",
      "message": "The Big Bad Wolf is at my brick house
                  demanding I open the door and threatening
                  to blow it down! Please come quickly!"
    }

[Python executes call_hunter(urgency="emergency", ...)]
  -> "The hunter is sprinting to your location with backup!"

[API Request sent — Round 2]
  messages: [ system, user, assistant(tool_calls), tool(result) ]

[API Response received — Round 2]
  finish_reason: stop
  content: "*Peeking through the window as the hunter charges
            forward*
            Thank goodness you're here! Wolf can't touch this
            brick now... *takes a shaky breath*
            I'll stand tall while you handle him!"

Pig: *Peeking through the window as the hunter charges forward*
     Thank goodness you're here! Wolf can't touch this brick
     now... I'll stand tall while you handle him!
```

Key observations:

- `finish_reason: tool_calls` — the model signalled it wants to call a function instead of generating a reply
- The Python host (not the model) executed `call_hunter()` and got a concrete result
- A **second API call** was required to let the model incorporate the tool result into its final answer
- The pig now genuinely acts rather than only speaking

What Changed When Tools Were Enabled

Aspect	Without tools	With tools
API request	`messages` only	`messages` + `tools` array
API response	Plain `content` string	`tool_calls` list
`finish_reason`	`stop`	`tool_calls` (first round)
Python role	Display text	Execute function, feed result back
Model round trips	1	2
Outcome	Verbal response only	Real action taken

Part 2 — How Function Calling Works

What It Is

When the model receives a conversation together with a list of tool schemas, it can decide — instead of writing a direct answer — to *request* that the host application call a specific function with specific arguments. The model outputs a structured JSON object (the `tool_calls` field) describing what it wants done.

The host program, **not** the model, executes the real Python function, then sends the result back through a `tool` message. The model receives this new context and produces its final human-readable answer. This is the complete agentic loop.

Normal Answer vs Tool Call

The difference is visible directly in the API response structure:

Normal response — model answers with text, conversation ends:

```
{
  "finish_reason": "stop",
  "message": {
    "role": "assistant",
    "content": "My bricks are too strong for your tricks..."
  }
}
```

Tool call response — model requests an action, conversation must continue:

```
{
  "finish_reason": "tool_calls",
  "message": {
    "role": "assistant",
    "content": null,
    "tool_calls": [{
      "id": "call_0_call_hunter_0",
      "type": "function",
      "function": {
        "name": "call_hunter",
        "arguments": "{\"urgency\": \"emergency\", \"message\": \"...\"}"
      }
    }]
  }
}
```


In the second case the `content` field is `null` — the model produced no text. It is waiting for the tool result before it can continue.

Why the Host Remains in Control

The model never executes any code. It only reads and writes text. Every real side-effect — filesystem writes, network calls, database queries — lives inside Python functions that the host chooses to call (or not). The host can validate arguments, rate-limit tool use, log calls, or refuse dangerous requests before anything is executed. This design gives predictability and safety that a model running arbitrary code cannot provide.

In the Three Pigs demo this means:

1. The model decides the hunter should be called and selects the urgency level
2. `three_pigs_function_calling.py` calls `call_hunter()` locally
3. Only the string result enters the model's context — the model never touches the Python function directly

---

Part 3 — Math Solver Design

Chosen Tools

Four tools were defined, matching the recommended minimum from the brief:

Tool	Description
`evaluate_expression(expression)`	Simplifies or evaluates any SymPy-compatible expression; returns exact form and decimal approximation
`solve_equation(equation, variable)`	Splits on = and solves for the given variable; returns all roots
`factor_expression(expression)`	Factors polynomials into irreducible integer factors
`plot_function(expression, x_min, x_max)`	Lambdifies the expression with NumPy, plots with Matplotlib, saves a PNG to `plots/`

Why the Tool Set Is Small

A small, focused tool set produces better model behaviour for two reasons:

1. **Disambiguation** — When fewer tools exist there is less chance the model chooses the wrong one. A mega-function called `do_math()` would require the model to use natural language arguments that are much harder to validate.
2. **Schema clarity** — Each JSON schema stays short and its `description` field is precise. The model reads these schemas as part of its context, so clarity directly maps to correct tool selection.

Adding tools for every conceivable operation (integration, limits, matrices ...) would create noise and increase the probability of hallucinated arguments.

Key Code Fragments

Tool implementations (SymPy)

```python
def evaluate_expression(expression: str) -> str:
    expr = sp.sympify(expression, evaluate=True)
    simplified = sp.simplify(expr)
    if simplified.is_number:
        if simplified.is_integer:
            return str(simplified)
        num_val = float(sp.N(simplified))
        return f"{simplified}  ≈  {num_val:.6g}"
    return str(simplified)

def solve_equation(equation: str, variable: str = "x") -> str:
    var = sp.Symbol(variable)
    lhs_str, rhs_str = equation.split("=", 1)
    expr = sp.sympify(lhs_str.strip()) - sp.sympify(rhs_str.strip())
    solutions = sp.solve(expr, var)
    return f"{variable} = " + ",  ".join(str(s) for s in solutions)
```

Tool schema (JSON)

```
{
  "type": "function",
  "function": {
    "name": "solve_equation",
    "description": "Solve an algebraic equation for a given variable and return
all solutions.",
    "parameters": {
      "type": "object",
      "properties": {
        "equation": { "type": "string",
                      "description": "e.g. 'x**2 - 5*x + 6 = 0'" },
        "variable": { "type": "string",
                      "description": "Variable to solve for (default: 'x')" }
      },
      "required": ["equation"]
    }
  }
}
```

Agentic loop

```
while True:
    response = client.chat.completions.create(
        model=model, messages=messages,
        tools=TOOLS, tool_choice="auto"
    )
    msg = response.choices[0].message
    messages.append({"role": "assistant", "content": msg.content or "",
                     "tool_calls": [...]})

    if not msg.tool_calls:          # final answer — print and exit loop
        display(msg.content)
        break

    for tc in msg.tool_calls:       # execute each requested tool
        result = TOOL_FUNCTIONS[tc.function.name]
(**json.loads(tc.function.arguments))
        messages.append({"role": "tool", "tool_call_id": tc.id, "content":
result})
```

The loop continues until the model produces a reply with no `tool_calls`. This allows multi-step problems where several tools are needed in sequence.

Plot function

```
def plot_function(expression, x_min=-10.0, x_max=10.0):
    x = sp.Symbol("x")
    f = sp.lambdify(x, sp.sympify(expression), modules=["numpy"])
    xs = np.linspace(float(x_min), float(x_max), 800)
    ys = np.asarray(f(xs), dtype=float)
```

```
    fig, ax = plt.subplots(figsize=(8, 5))
    ax.plot(xs, ys, linewidth=2, color="#1f77b4")
    ax.set_title(f"f(x) = {expression}")
    ax.grid(True, linestyle="--", alpha=0.4)
    safe = "".join(c if c.isalnum() or c in "._+-" else "_" for c in expression)
    path = PLOTS_DIR / f"{safe}.png"
    fig.savefig(path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return str(path)
```

`sp.lambdify` converts the symbolic SymPy expression into a callable NumPy function so that vectorised evaluation across 800 x-points is fast and accurate.

Part 4 — Testing Evidence

Algebra

Linear equation — `Solve 2x + 5 = 17`

```
→ solve_equation(equation='2*x + 5 = 17', variable='x')
← x = 6
```

The model correctly translates the natural-language equation into SymPy syntax, calls the tool, and returns a step-by-step verification.

Quadratic equation — `What are the roots of x^2 - 5x + 6 = 0?`

```
→ solve_equation(equation='x**2 - 5*x + 6 = 0', variable='x')
← x = 2,  3
```

Both roots returned. The final answer included the factored form `(x-2)(x-3)` as a cross-check.

Factorization — `Factor x^2 + 7x + 12`

```
→ factor_expression(expression='x**2 + 7*x + 12')
← (x + 3)*(x + 4)
```

Arithmetic

Fractions — `Evaluate (3/4 + 2/3) * 6`

```
→ evaluate_expression(expression='(3/4 + 2/3) * 6')
← 17/2  ≈  8.5
```

SymPy returns the exact rational `17/2` and the decimal approximation.

Plotting

Plot a parabola — `Plot y = x^2 - 4x + 3 from x = -2 to x = 6`

```
→ plot_function(expression='x**2 - 4*x + 3', x_min=-2, x_max=6)
← .../plots/x__2_-_4_x_+_3.png
```

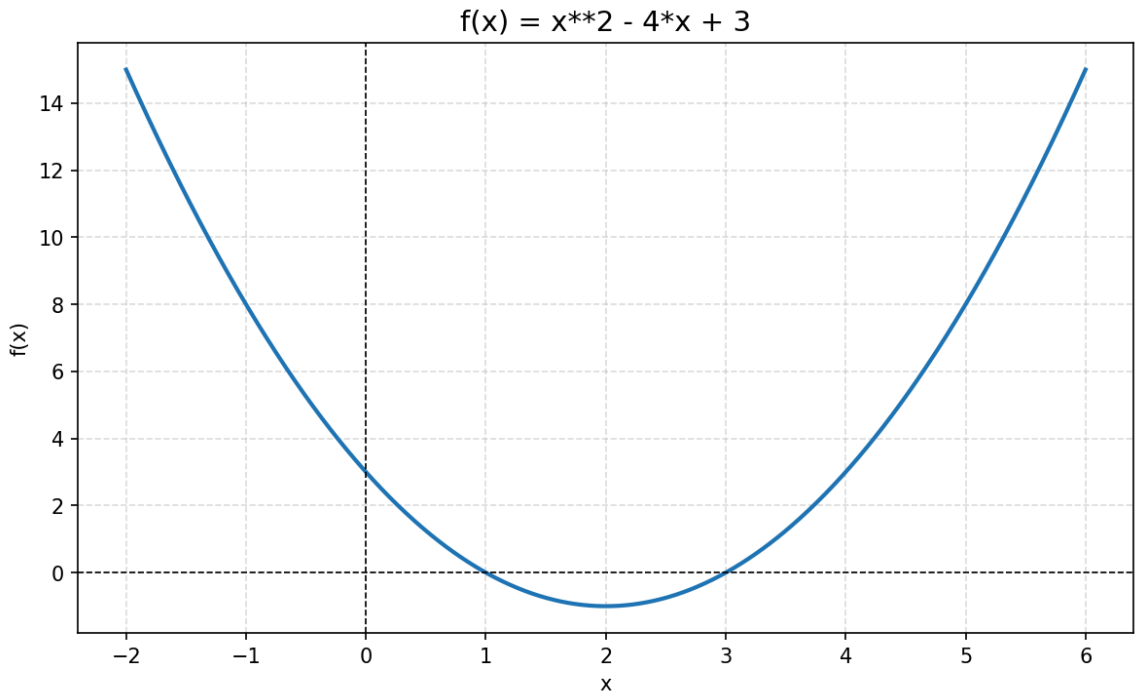

Figure 1: Plot of $x^2 - 4x + 3$

Vertex and plot — `What is the vertex of y = x^2 - 6x + 5? Plot it too`

The model called three tools in a single turn:

```
→ evaluate_expression(expression='-(-6)/(2*1)')
← 3
→ evaluate_expression(expression='3**2 - 6*3 + 5')
← -4
→ plot_function(expression='x**2 - 6*x + 5', x_min=-2, x_max=8)
← .../plots/x__2_-_6_x_+_5.png
```

Final answer: vertex at **(3, −4)**.

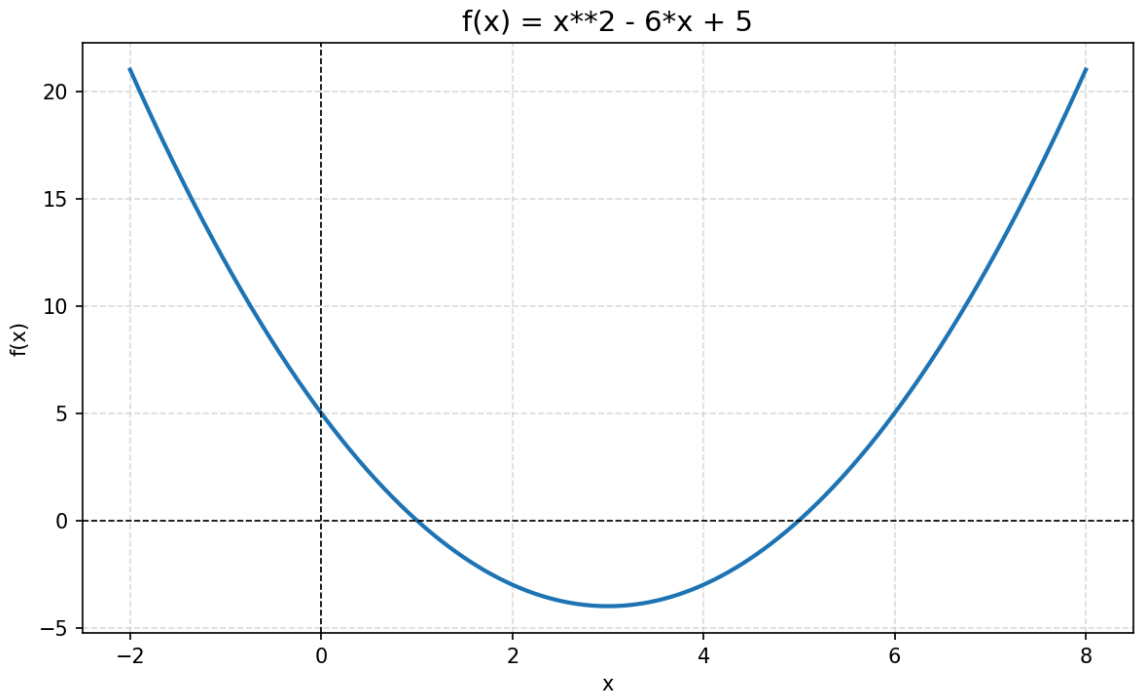

Figure 2: Plot of x^2 - 6x + 5

Robustness / Failure Cases

Invalid syntax — `Solve x^@# broken = 0`
The model recognised the invalid characters before calling any tool and responded:

> *I notice there seems to be some corruption in your equation — it shows "x^@# broken = 0" which contains invalid characters. Could you provide the correct equation?*

No tool was called and the program continued without crashing.

Division by zero — `Plot the function sin(x) / 0`
The model again declined to call the tool:

> *The function sin(x) / 0 involves division by zero, which is undefined in mathematics. You cannot divide any number by zero…*

The program remained stable and offered alternative suggestions.

Part 5 — Reflection

What the Model Did Well

- **Tool selection**: In every valid test the model chose the correct tool on the first attempt without any ambiguity.
- **Argument translation**: It reliably converted human notation (e.g. `x^2`) into SymPy syntax (`x**2`) in tool arguments.
- **Multi-tool chains**: For the vertex problem it planned and executed three tool calls in sequence to compute `x_vertex`, `y_vertex`, and then plot — all within a single agent loop iteration.
- **Failure gracefully**: For both invalid inputs the model reasoned about the problem before calling any tool, avoiding unnecessary error messages from SymPy.

Where It Fell Short

- **Implicit tool need**: For the vertex question the model used `evaluate_expression` to compute `-b/2a` arithmetically instead of a dedicated vertex function. This worked but is roundabout.
- **Over-explaining**: The final answers were sometimes longer than necessary for a high-school student. A tighter system prompt would help.
- **No simplification hint**: When the model suggested solutions to the failed parses, it offered generic polynomial examples rather than trying to interpret what the user *might* have meant.

LLMs as Orchestrators, Not Calculators

The key insight from this exercise is that the model should never be trusted to *compute* — it should be trusted to *decide*. When asked `(3/4 + 2/3) * 6`, a raw LLM might confidently return a wrong decimal. With function calling the model instead identifies that an evaluation is needed, constructs a valid SymPy expression, and lets the deterministic Python function produce the correct answer. The model acts as an intelligent dispatcher; the Python code acts as the reliable executor.

This separation is what makes function-calling agents more reliable than pure-text generation for any task requiring precision.

Part 6 — Questions

1. Why is function calling more reliable than asking the model to "just do the math" in plain text?

LLMs are probabilistic text predictors. They can produce plausible-looking but wrong arithmetic. A call to `sp.solve()` or `sp.simplify()` is deterministic — given the same input it always gives the same correct result. Function calling routes numerical work to code that cannot hallucinate.

2. Why should the available tool set be small and well-defined?

Each tool adds to the decision space the model must reason over. Vague or overlapping tools increase the probability of the wrong tool being chosen or arguments being malformed. A small set with precise descriptions and non-overlapping responsibilities consistently produces correct choices.

3. What is the role of SymPy in the solution?

SymPy provides symbolic algebra: exact rational arithmetic, algebraic simplification, polynomial factoring, and symbolic equation solving. It replaces floating-point approximations with mathematically exact results that can be displayed as fractions or radicals.

4. What is the role of Matplotlib in the solution?

Matplotlib renders the NumPy array of `(x, f(x))` pairs produced by `sp.lambdify` into a publication-quality PNG. The file is a tangible, persistent artefact the student can open and inspect, which a text-only answer could never provide.

5. What happens from the moment the user types a problem to the final answer?

1. The input is added to `messages` as a `user` turn.
2. `client.chat.completions.create()` sends `messages` + `TOOLS` to the model.
3. If the response contains `tool_calls`, each call is dispatched to the matching Python function.
4. Each result is appended as a `tool` message and the API is called again.
5. When the model responds with text only (no `tool_calls`), the answer is displayed and the loop exits.

6. What kinds of errors can still happen even when function calling is used?

- The model may pass syntactically invalid SymPy expressions (e.g. using ^ instead of **).
- It may call the wrong tool (e.g. `evaluate_expression` instead of `solve_equation`).
- Tool functions can raise `ValueError` or `TypeError` if the expression is mathematically undefined.
- The model may hallucinate a tool name that does not exist in `TOOL_FUNCTIONS`.

All of these are handled either by the `try/except` blocks in each tool function (which return an error string rather than crashing) or by the model's own reasoning before it calls the tool.

7. When should the model answer directly, and when should it call a tool?

The model should answer directly when the question is conceptual, definitional, or requires only language (e.g. "What is a quadratic equation?"). It should call a tool whenever the answer requires computation, exact algebra, or a visual output that would be error-prone to produce from memory.

Part 7 — How to Run the Code

The project lives in `week-03/math-solver/`.

```
cd week-03/math-solver
uv sync
# Add your credentials to .env (see apikey.md — do not commit secrets)
python math_solver.py
```

Main file: `math_solver.py`
Plots are saved to: `math-solver/plots/`

Dependencies (managed by `uv` / `pyproject.toml`):

```
openai, python-dotenv, sympy, matplotlib, numpy, rich
```